pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Portfolio Aggregation and Scenarios

WEEK 3 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

From 200 bonds to one book

The September book, holdings.csv: 200 bonds as of 30 September 2026. Five questions:

1. **What is the book worth, and each position's share?**
2. **Book duration, spread, DTS, and DV01**
3. **The split by sector and rating bucket**
4. **Rate and spread scenarios, full reprice**
5. **Duration plus convexity against the reprice**

Every scenario is a mechanical what-if about a synthetic book. Not a forecast.

Week 3: curve, spread, and the book

Day 11	Treasury curve interpolation
Day 12	Spread over the curve
Day 13	Spread duration and DTS
Day 14	**Aggregation and scenarios**
Day 15	The capstone

Market value: par times the dirty price

```
market value = par_held x (price + accrued) / 100
```

Bezane Finance	Value
Par held, dollars	5,000,000
Clean + accrued	106.727 + 1.983
Dirty price	108.710
Market value, dollars	5,435,500.00

All 200 tie to the file to the cent. The book: 490,819,215.00 dollars.

Excel side by side (tested in Excel)

`=R2*(I2+K2)/100`	5,435,500.00
`=SUMPRODUCT(R2:R201,I2:I201+K2:K201)/100`	490,819,215.00
`=SUM(S2:S201)`	490,819,215.00

holdings.csv opened in Excel: R par_held, I price, K accrued, S market_value.

weighted_average: SUMPRODUCT over SUM

```
# sum of value x weight / sum of weights
total = 0.0
for value, weight in zip(values, weights):
    total += value * weight
return total / total_weight
```

Excel side by side (tested in Excel)

`=SUMPRODUCT(N2:N201,S2:S201)/SUM(S2:S201)`	6.254777
`=AVERAGE(N2:N201)`	6.470405

Values first, weights second: the order weighted_average takes them in.

Weights that sum to zero raise ValueError: the weights sum to zero.

6.254777

years: market-value weighted duration

6.470405

years: plain average of 200 durations

The book in one set of numbers

6.2548

years: book duration

180.46

bp: book spread

1064.11

years x bp: book DTS

306,996.11

dollars per bp: book DV01

Duration, spread, DTS, and convexity are averaged. Dollar DV01 is added up.

Every bond recomputed from its yield with bondmath: dirty price, modified duration, convexity, DV01, spread duration, DTS. Durations match the file's rounded column.

Excel side by side (tested in Excel)

`=SUMPRODUCT(N2:N201,S2:S201)/10000`	306,996.49

By sector and by rating bucket

Bucket	Weight %	Spread bp	DTS	DTS contrib.
AAA	1.67	21.0	116.7	1.95
AA	2.66	34.1	150.5	4.00
A	30.26	77.8	507.2	153.45
BBB	30.80	129.2	965.1	297.24
BB	23.18	219.5	1261.9	292.47
B	8.88	485.8	2589.9	230.04
CCC	2.56	854.2	3321.9	84.95

BB, B, and CCC: 34.62 percent of market value, 57.1 percent of DTS.

Excel side by side (tested in Excel)

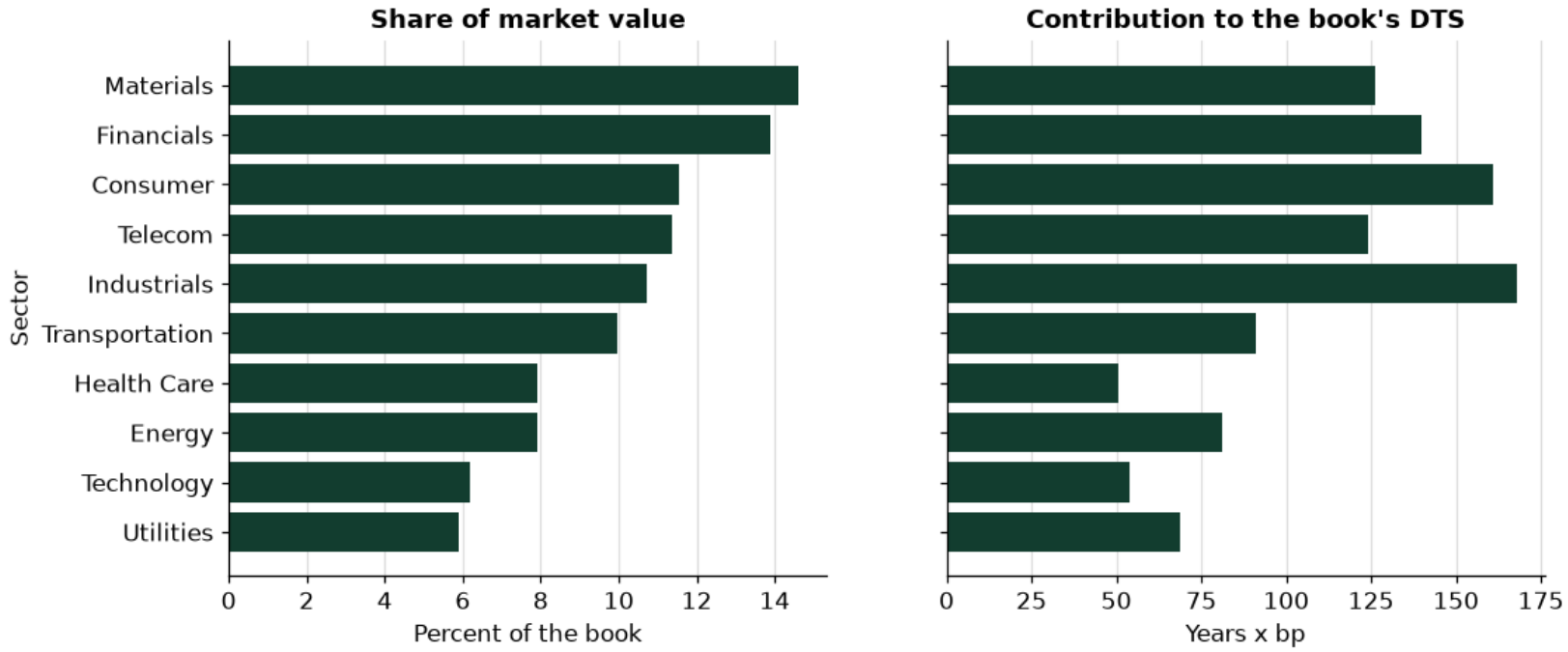
Helper column mv_x_duration =S2*N2. Insert, PivotTable: sector in Rows, Sum of market_value in Values. Then a calculated field:

```
=mv_x_duration/market_value
```

It works on the sums, so it gives the weighted average: 5.263418 for Materials, 6.254777 on the Grand Total.

A description of the synthetic book, not a view on any sector.

Sector: value and DTS



A sector can hold a modest share of the value and a larger share of the DTS. A description of a synthetic book, not a view on any sector.

Rate scenarios: parallel shifts, full reprice

A mechanical what-if: instantaneous, no carry, no roll-down, not a forecast

Scenario	Change, dollars	Percent
rates -100 bp	32,430,983.84	6.608
rates -25 bp	7,778,969.05	1.585
rates +25 bp	-7,573,434.14	-1.543
rates +100 bp	-29,134,835.97	-5.936

Down 100 gains more than up 100 loses: convexity.

Excel side by side (tested in Excel)

Y1 holds the shift in bp. V2, filled down:

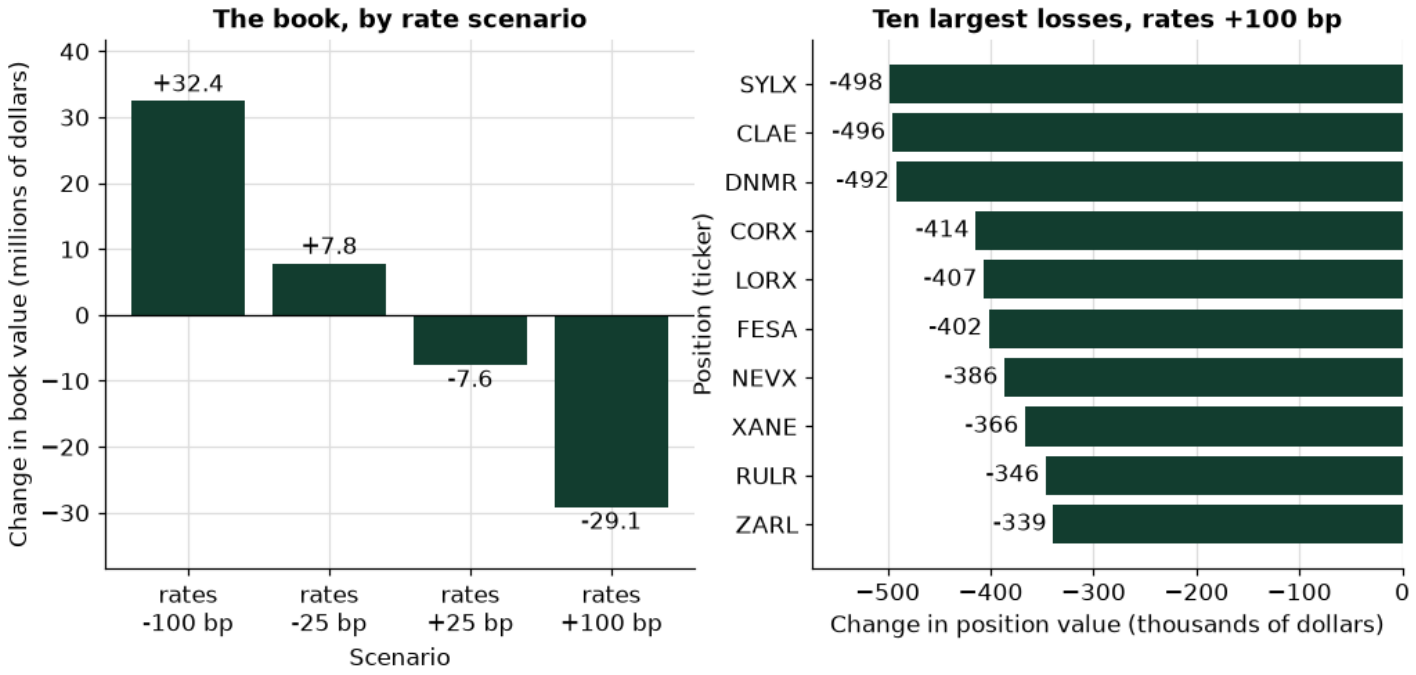
```
=PRICE(A2,H2,G2/100,(L2+$Y$1/100)/100,100,2,0)
```

W2 =R2*(V2+K2)/100 filled down; AA2 =SUM(W2:W201). Shifts in Z3:Z7, select Z2:AA7, Data, What-If Analysis, Data Table, Column input cell Y1.

```
=TABLE(,Y1)
```

At +100: -29,134,835.97 dollars, the same as Python to the cent.

The book under the rate scenarios



Down 100 gains more than up 100 loses: convexity. The ten losses shown are a minority of the total. A what-if, not a forecast.

Spread scenarios: two shapes

Every spread up 25 bp: the same move for every bond

Every spread up 10 percent: wide spreads move most

Scenario	Change, dollars	Percent
spreads +10 percent	-5,150,093.57	-1.049
spreads +25 bp	-7,573,434.14	-1.543

Up 25 bp equals the +25 bp rate scenario exactly: one curve plus a spread.

Bucket	Spread	Move, +10%	Loss
CCC	854 bp	85 bp	3.24%
BBB	129 bp	13 bp	0.96%

-1.0641%

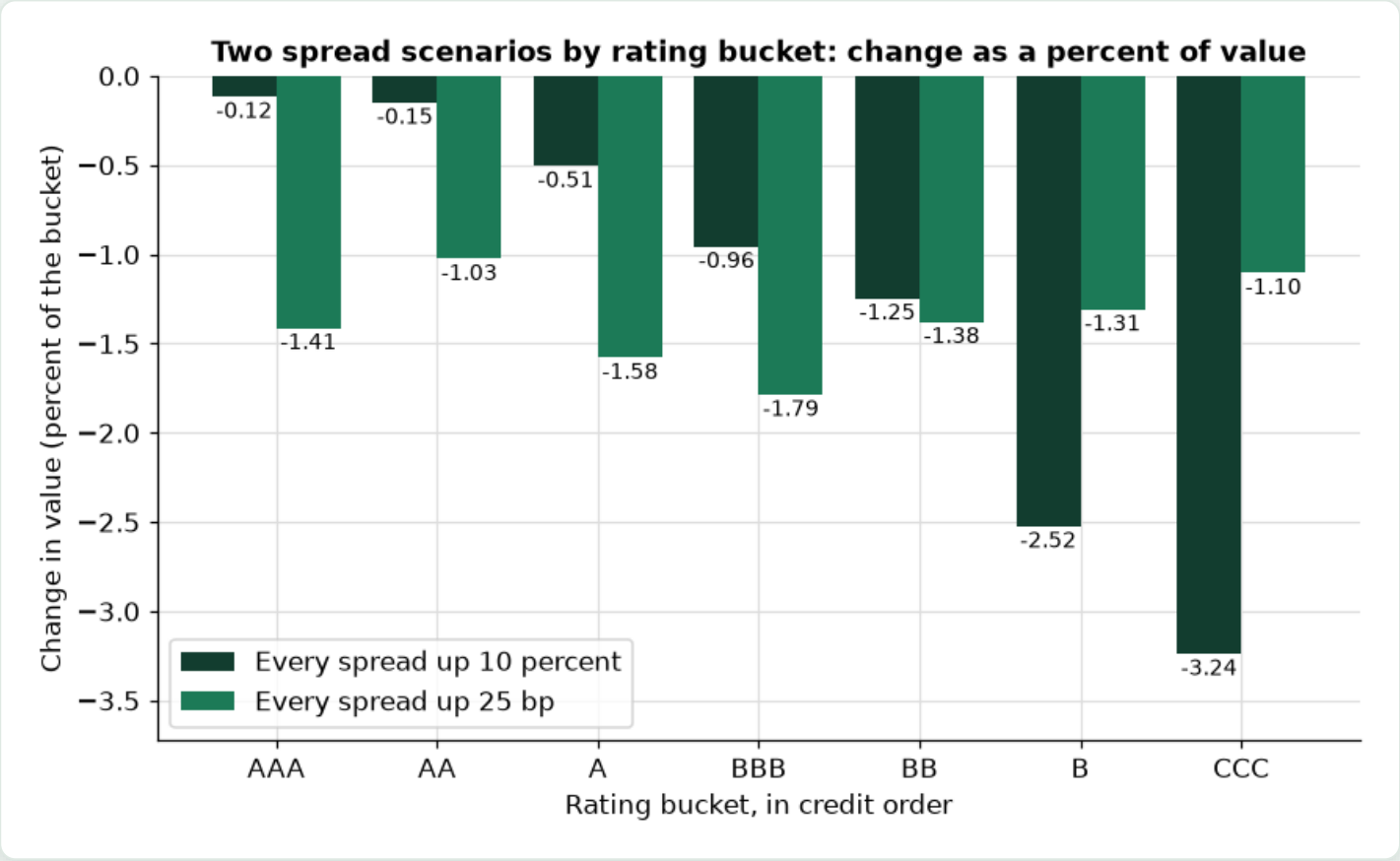
from book DTS, up 10 percent

-1.0493%

full reprice, up 10 percent

DTS times a relative spread change: a first-order estimate.

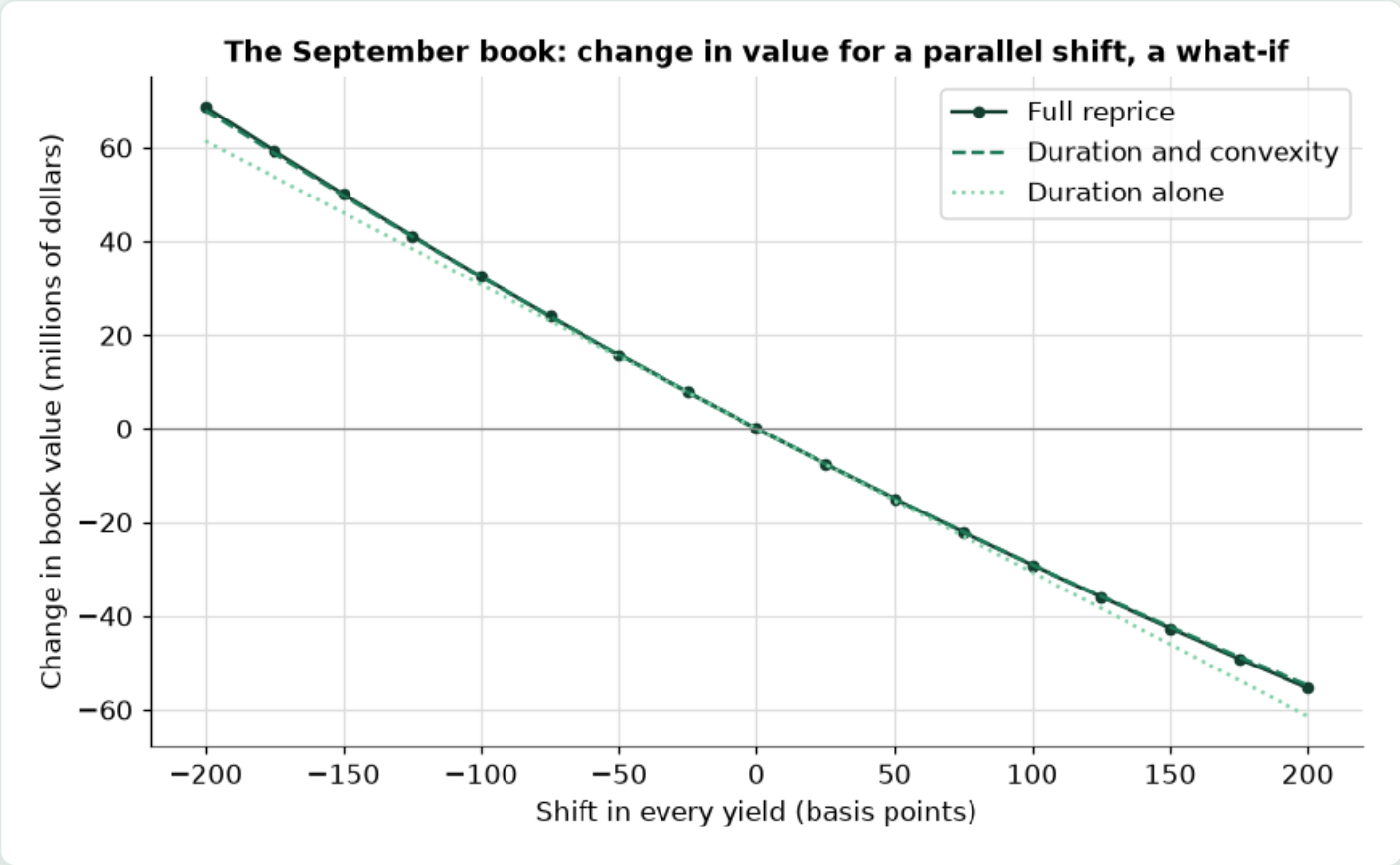
Two spread scenarios by bucket



Green: every spread up 25 bp, which follows each bucket's duration. Dark: every spread up 10 percent, which grows toward CCC.

Same book, two shapes of move, two pictures. A report should say which shape it used.

Full reprice against price_change_estimate



Scenario	Gap, dollars
rates -100 bp	87,345.99
rates -25 bp	1,314.62
rates +25 bp	-1,283.05
rates +100 bp	-79,251.74

Bezane at +100 bp: full -4.8897, with convexity -4.8867, duration alone -5.0300, per 100.

For a large scenario, quote the full reprice.

The scenario grid in a workbook

```
with pd.ExcelWriter("scenario_grid.xlsx") as writer:
    grid.to_excel(writer, sheet_name="scenarios",
                  index=False)
    by_sector_scenarios.to_excel(writer,
                  sheet_name="by_sector")
```

pd.read_excel reads both sheets back: the same names and sectors, the numbers to within billionths of a dollar. Values, not formulas, so pandas has something to read.

What Excel shows (tested in Excel)

Two sheets, scenarios and by_sector. Thousands separators and cents, three decimals for pnl_pct, no ####.

rates +100 bp, pnl_dollars:

-29,134,835.97

Click a cell: the formula bar shows every digit.

Today's Excel, every formula tested

Job	Excel (tested)	Returns
Row market value	`=R2*(I2+K2)/100`	5,435,500.00
Book market value	`=SUMPRODUCT(R2:R201,I2:I201+K2:K201)/100`	490,819,215.00
Book duration	`=SUMPRODUCT(N2:N201,S2:S201)/SUM(S2:S201)`	6.254777
Book spread	`=SUMPRODUCT(M2:M201,S2:S201)/SUM(S2:S201)`	180.4610
Book DV01	`=SUMPRODUCT(N2:N201,S2:S201)/10000`	306,996.49
By sector	`pivot, calculated field =mv_x_duration/market_value`	5.263418
Rate scenarios	`Data Table over =SUM(W2:W201), input Y1`	-29,134,835.97

All on holdings.csv opened in Excel. Python gives the same numbers.

Unstage a file added by mistake

git add … scenario_grid.xlsx

```
M  bondmath.py
A  scenario_grid.xlsx
M  test_bondmath.py
```

git restore --staged

```
git restore --staged
    scenario_grid.xlsx
git status --short
```

After

```
M  bondmath.py
M  test_bondmath.py
?? scenario_grid.xlsx
```

The workbook is an output: the notebook makes it again on every run, and Git cannot show a readable diff of a binary file. The record holds the code that makes it.

--staged changes only the staging area. The file stays in the folder; nothing is deleted.

Bring it home: paste, test, check, commit

1

Your folder

Set-Location to my_bondmath, in your projects folder.

2

Paste

Today's two functions and two test cells, without the %%writefile line.

3

Test

python -m pytest -q: 40 passed, or 53 with your exercise functions.

4

Commit

Read the diff, stage the two files, check the status, and commit.

```
git restore --staged <file name>
```

The habit: a file staged by mistake comes back out before the commit. In Colab, download the two files, and the workbook if you want it in Excel.

The AI check

The docstring is the prompt. The code that comes back runs, and has one convention error.

1

Docstring

Units, convention, and the Excel formula: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert against Excel's reference rows. Run it: it fails.

→

4

Fix

Find the convention it broke, fix it, run the test again.

Today: market value uses the clean price, so the weights leave out accrued interest.

Code that runs is not the same as code that is right.

What you can do now

- Tie market value to par x dirty price, to the cent
- Weight by market value: weighted_average, SUMPRODUCT over SUM
- Book duration, spread, DTS, and DV01, by sector and rating
- Rate and spread scenarios by full reprice, as what-ifs
- price_change_estimate, a workbook, and git restore --staged

Week 3, Day 5 The capstone: clean a November book, reconcile it, and write a risk report